Dokumentasi Praktikum Pemrograman Mobile

Pertemuan 3



Nama:

Muhammad Yusuf Bagus Syafaat

NIM:
20240801041

UNIVERSITASA ESA UNGGUL TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

2026

1. Pendahuluan

   Pada praktikum ini dibahas penggunaan TextField, TextEditingController, serta Form dan TextFormField untuk menerima dan memvalidasi input dari pengguna. Validasi diperlukan agar data yang dimasukkan sesuai dengan ketentuan sebelum diproses oleh aplikasi. Selain itu, praktikum memperkenalkan konsep state, yaitu data yang dapat berubah dan memengaruhi tampilan aplikasi.

   Pengelolaan state pada aplikasi sederhana dapat dilakukan menggunakan setState. Namun, ketika data perlu digunakan oleh beberapa widget atau halaman, pengelolaannya dapat menjadi lebih kompleks. Oleh karena itu, pada praktikum ini digunakan ChangeNotifier dan Provider sebagai dasar state management.

2. Tujuan Praktikum
   - Memahami penggunaan TextField dan TextEditingController.
   - Membuat form dengan validasi.
   - Memahami perbedaan setState dan state management.
   - Mengimplementasikan ChangeNotifier dan Provider.
   - Memahami penggunaan context.watch() dan context.read().

3. Teori dasar
   - Input dan form. TextField menerima teks; nilainya dibaca lewat TextEditingController. Untuk beberapa isian yang perlu divalidasi sekaligus, gunakan Form dengan GlobalKey<FormState> lalu panggil validate().
   - State adalah data yang dapat berubah dan memengaruhi tampilan.
   - Provider menempatkan objek state (turunan ChangeNotifier) di atas pohon widget. Widget yang membutuhkannya cukup "berlangganan"; ketika model memanggil notifyListeners(), widget tersebut dibangun ulang otomatis.

4. Langkah praktikum

   Buat projek baru flutter di terminal:

   ```
   flutter create praktikum1
   cd praktikum3
   flutter run
   ```

5. Implementasi textfield

   Pada tahap pertama, dibuat halaman input sederhana yang memungkinkan pengguna memasukkan nama. Nilai input disimpan menggunakan TextEditingController, kemudian ditampilkan sebagai sapaan ketika tombol ditekan. Modul menggunakan StatefulWidget karena terdapat data _salam yang berubah setelah tombol ditekan.

```dart
class _InputPageState extends State<InputPage> {
  final _controller = TextEditingController();
  String _salam = '';

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }
```

_controller digunakan untuk mengambil teks yang dimasukkan pengguna melalui TextField. Sementara itu, _salam digunakan untuk menyimpan teks sapaan yang akan ditampilkan pada halaman. Method dispose() digunakan untuk melepaskan TextEditingController ketika halaman tidak lagi digunakan.

Selanjutnya, input nama dibuat menggunakan TextField:

```dart
TextField(
  controller: _controller,
  decoration: const InputDecoration(
    labelText: 'Nama',
    border: OutlineInputBorder(),
  ),
```

Kode tersebut menghubungkan TextField dengan _controller. labelText memberikan keterangan bahwa field digunakan untuk memasukkan nama, sedangkan OutlineInputBorder memberikan bentuk garis pada input.

6. Implementasi form dengan validasi

   Tahap berikutnya adalah membuat form yang memiliki beberapa input, yaitu nama lengkap, email, jurusan, dan persetujuan pengguna. Modul menggunakan Form dengan GlobalKey<FormState> agar seluruh input dapat divalidasi ketika tombol Daftar ditekan.

```dart
class _FormPageState extends State<FormPage> {
  final _formKey = GlobalKey<FormState>();
  final _nama = TextEditingController();
  final _email = TextEditingController();

  String? _jurusan;
  bool _setuju = false;
```

- _formKey digunakan untuk mengidentifikasi form.
- _nama digunakan untuk mengontrol input nama.

- _email digunakan untuk mengontrol input email.
- _jurusan menyimpan pilihan jurusan.
- _setuju menyimpan status checkbox persetujuan.

Jika seluruh validasi berhasil, aplikasi menampilkan SnackBar yang berisi nama dan jurusan pengguna.

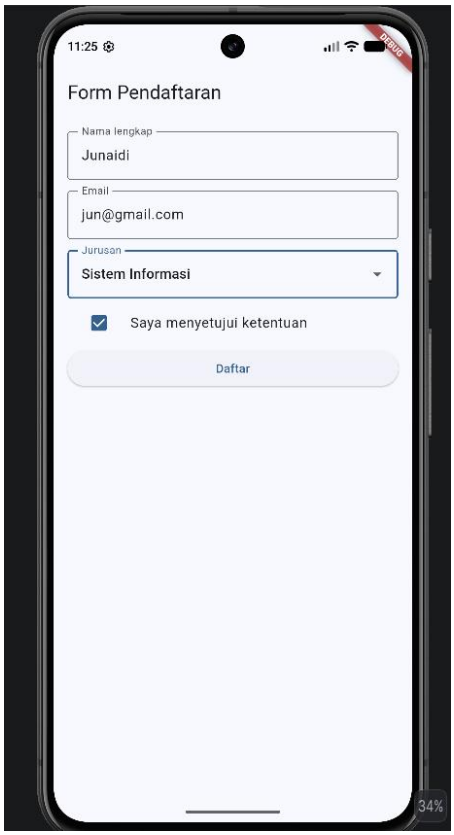
```
TextFormField(
  controller: _nama,
  decoration: const InputDecoration(
    labelText: 'Nama lengkap',
    border: OutlineInputBorder(),
  ), InputDecoration
  validator: (v) =>
  (v == null || v.trim().isEmpty)
      ? 'Nama wajib diisi'
      : null,
), TextFormField
```

Pada bagian validator, aplikasi memeriksa apakah input nama kosong. Jika kosong, validator mengembalikan pesan "Nama wajib diisi". Jika input valid, validator mengembalikan null.

```
TextFormField(
  controller: _email,
  keyboardType: TextInputType.emailAddress,
  decoration: const InputDecoration(
    labelText: 'Email',
    border: OutlineInputBorder(),
  ), InputDecoration
  validator: (v) {
    if (v == null || !v.contains('@')) {
      return 'Email tidak valid';
    }
    return null;
  },
```

Validasi email pada praktikum dilakukan dengan memeriksa apakah input mengandung karakter @. Jika tidak ditemukan, aplikasi menampilkan pesan "Email tidak valid".

Hasil akhir:



7. Mengapa butuh State Management?
   Pada Bagian A dan B, state hidup di dalam satu halaman (setState). Bayangkan aplikasi daftar tugas: halaman daftar menampilkan tugas, sedangkan halaman tambah membuat tugas baru. Kedua halaman perlu berbagi data yang sama. Mengoper data lewat constructor dan callback dari satu halaman ke halaman lain cepat menjadi rumit. Solusinya: pindahkan data ke satu objek bersama yang dapat diakses kedua halaman. Inilah peran Provider.
8. Implementasi provider
   Sebelum menggunakan provider, di terminal folder proyek, pasang paket dengan perintah:

   flutter pub add provider

```
PS C:\Users\LENOVO LOQ\pemmob\pert3\praktikum3> flutter pub add provider
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
+ nested 1.0.0
+ provider 6.1.5+1
  test_api 0.7.12 (0.7.14 available)
```

```dart
class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

  List<Tugas> get items => List.unmodifiable(_items);

  int get jumlahSelesai =>
      _items.where((t) => t.selesai).length;
```

Pada code ini, items berfungsi menyimpan daftar tugas, items mengambil data tugas, jumlahSelesai menghitung tugas yang sudah selesai, dan ChangeNotifier yang memungkinkan widget mendapatkan pemberitahuan ketika state berubah.

```dart
  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }
}
```

Ketiga method tersebut menggunakan notifyListeners() agar perubahan state dapat diteruskan ke widget yang menggunakannya.

tambah() :Menambahkan tugas
toggle() :Mengubah status selesai/belum selesai
hapus() :Menghapus tugas
notifyListeners() :Memberitahu widget bahwa data berubah

```dart
void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => TugasModel(),
      child: const MyApp(),
    ), ChangeNotifierProvider
  );
}
```

ChangeNotifierProvider menyediakan TugasModel sehingga dapat diakses oleh halaman-halaman di dalam aplikasi.

```dart
void _simpan() {
  final judul = _controller.text.trim();

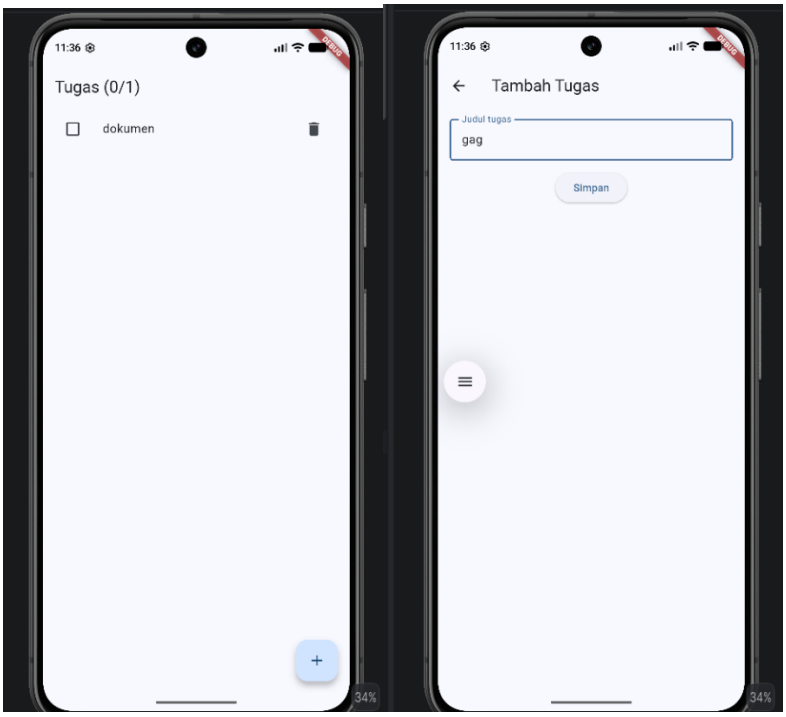
  if (judul.isEmpty) return;

  context.read<TugasModel>().tambah(judul);

  Navigator.pop(context);
}
```

- Mengambil teks dari input.
- Memastikan judul tidak kosong.
- Menambahkan data melalui TugasModel.
- Kembali ke halaman daftar setelah data disimpan.

Hasil akhir:

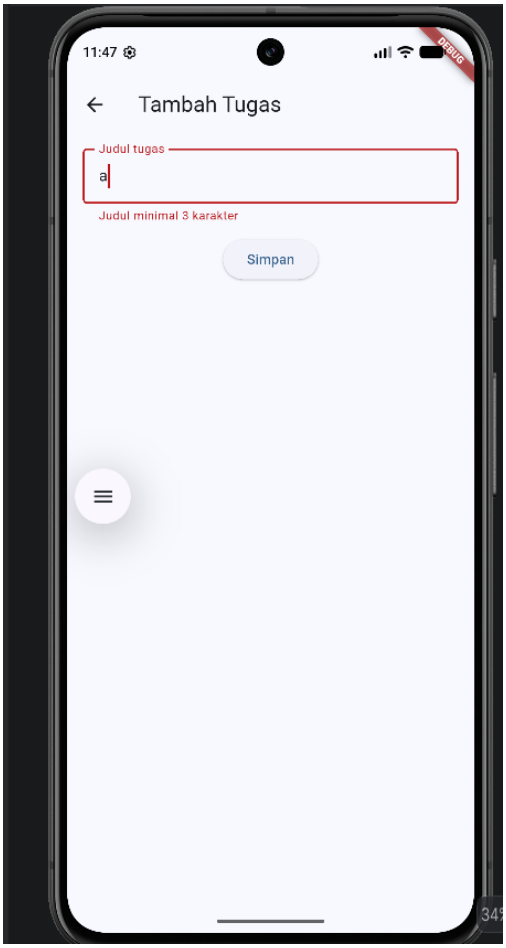


9. Latihan Mandiri
    a. Tambahkan validasi pada TambahPage: judul minimal 3 karakter dan tampilkan pesan error.

```
body: Form(
  key: _formKey,
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      children: [
        TextFormField(
          controller: _controller,
          autofocus: true,
          decoration: const InputDecoration(
            labelText: 'Judul tugas',
            border: OutlineInputBorder(),
          ), InputDecoration
          validator: (v) {
            if (v == null || v.trim().length < 3) {
              return 'Judul minimal 3 karakter';
            }

            return null;
          },
```

Di sini kalau input kosong atau panjangnya kurang dari 3 karakter, validator mengembalikan pesan "Judul minimal 3 karakter" yang otomatis muncul di bawah kolom. Kalau valid, validator mengembalikan null, artinya tidak ada error. Hasil:



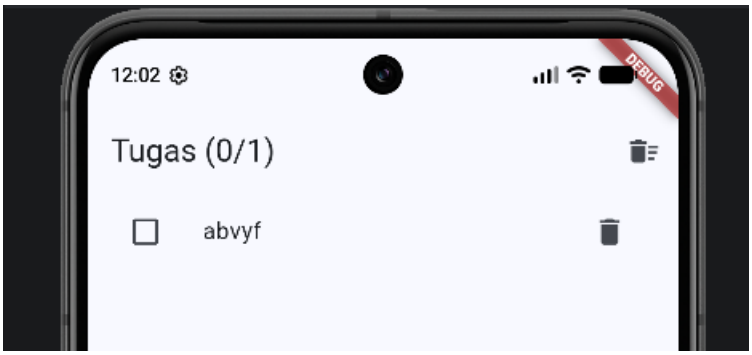
b. Tambahkan method hapusSelesai() pada TugasModel dan tombol di AppBar untuk menghapus semua tugas yang sudah selesai.

```
  void hapusSelesai() {
    _items.removeWhere((t) => t.selesai);
    notifyListeners();
  }
}
```

Method ini menggunakan removeWhere untuk menghapus semua tugas yang statusnya selesai sekaligus. Setelah dihapus, notifyListeners() dipanggil agar UI otomatis diperbarui.

```
appBar: AppBar(
  title: Text(
    'Tugas (${model.jumlahSelesai}/${model.items.length})',
  ), Text
  actions: [
    IconButton(
      icon: const Icon(Icons.delete_sweep),
      onPressed: () {
        context.read<TugasModel>().hapusSelesai();
      },
    ), IconButton
  ],
), AppBar
```

Actions di AppBar menambahkan tombol di sisi kanan atas, saat ditekan, nPressed memanggil context.read<TugasModel>().hapusSelesai() yang langsung menghapus semua tugas yang selesai.



Jadi, ketika beberapa tugas dipilih sekaligus lalu user klik icon tong sampah di paling atas, semua tugas yang dipilih otomatis dihapus.

c. Tampilkan SnackBar "Tugas ditambahkan" setelah tugas disimpan.

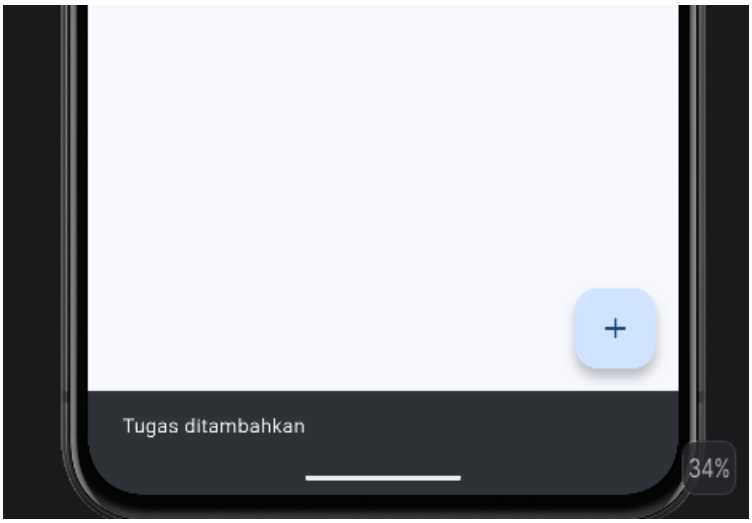
```
void _simpan() {
  if (_formKey.currentState!.validate()) {
    final judul = _controller.text.trim();

    context.read<TugasModel>().tambah(judul);

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Tugas ditambahkan'),
      ), SnackBar
    );

    Navigator.pop(context);
  }
}
```

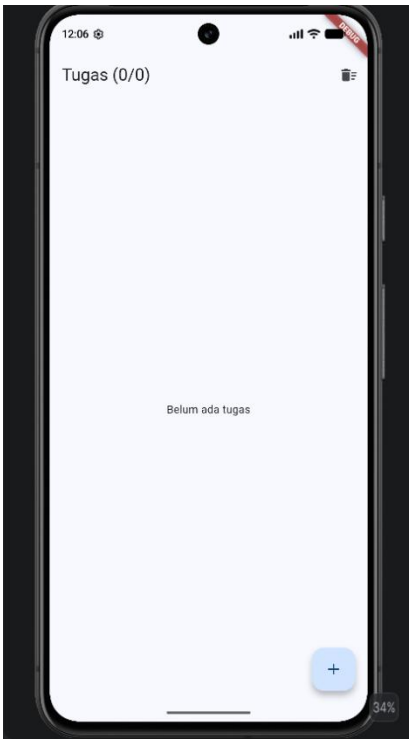
Disini, ketika sebuah tugas baru ditambahkan, sebuah snackbar otomatis muncul di layar, navigator pop membuat kembali ke halaman sebelumnya. Hasil:



d. Bila daftar kosong, tampilkan teks "Belum ada tugas" di tengah layar.

```
body: model.items.isEmpty
    ? const Center(
  child: Text('Belum ada tugas'),
)  Center
```

Disini jika tidak ada item di layar utama, pesan belum ada tugas otomatis muncul di tengah layar dan akan menghilang jika membuat tugas baru.



10. Buat aplikasi Daftar Belanja dengan ketentuan:
    a. Halaman Form Tambah: nama barang (wajib), jumlah (wajib, angka lebih dari 0), dan kategori (dropdown), dengan validasi pada setiap isian.
    b. Halaman Daftar: menampilkan semua barang, dapat dicentang sebagai "sudah dibeli" dan dihapus.
    c. State disimpan pada satu ChangeNotifier dan dibagikan dengan Provider; AppBar menampilkan jumlah barang yang belum dibeli.

11. Implementasi Tugas

```dart
class BelanjaModel extends ChangeNotifier {
  final List<Barang> _items = [];

  List<Barang> get items => List.unmodifiable(_items);

  int get jumlahBelumDibeli =>
      _items.where((barang) => !barang.sudahDibeli).length;

  void tambah(String nama, int jumlah, String kategori) {
```

Bagian ini digunakan sebagai pusat pengelolaan data pada aplikasi. BelanjaModel menggunakan ChangeNotifier untuk menyimpan daftar barang yang ditambahkan oleh pengguna. items digunakan untuk mengambil daftar barang, sedangkan jumlahBelumDibeli digunakan untuk menghitung jumlah barang yang belum ditandai sebagai sudah dibeli.

```
  void tambah(String nama, int jumlah, String kategori) {
    _items.add(
      Barang(
        nama,
        jumlah,
        kategori,
      ),
    );
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].sudahDibeli =
    !_items[index].sudahDibeli;

    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
    notifyListeners();
  }
}
```

Bagian ini berisi fungsi utama untuk mengelola daftar belanja. Fungsi tambah() digunakan untuk menambahkan barang, toggle() untuk mengubah status barang menjadi sudah atau belum dibeli, sedangkan hapus() digunakan untuk menghapus barang. notifyListeners() memastikan perubahan data langsung diperbarui pada tampilan aplikasi.

```
  runApp(
    ChangeNotifierProvider(
      create: (_) => BelanjaModel(),
      child: const MyApp(),
    ), ChangeNotifierProvider
  );
}
```

digunakan untuk menyediakan BelanjaModel agar dapat diakses oleh berbagai halaman dan widget dalam aplikasi. Dengan Provider, pengelolaan state tidak perlu dilakukan secara terpisah pada setiap halaman.

```
class _TambahBarangPageState
    extends State<TambahBarangPage> {
  final _formKey = GlobalKey<FormState>();
```

```
void _simpan() {
  if (_formKey.currentState!.validate()) {
    context.read<BelanjaModel>().tambah(
      _nama.text.trim(),
      int.parse(_jumlah.text),
      _kategori!,
    );

    Navigator.pop(context);
  }
}
```

Bagian ini digunakan untuk memproses data dari form tambah barang. GlobalKey<FormState> digunakan untuk menjalankan validasi pada setiap input. Jika seluruh input valid, data akan dikirim ke BelanjaModel melalui fungsi tambah(), kemudian halaman form ditutup dengan Navigator.pop(). Validasi jumlah memastikan data tidak kosong, harus berupa angka, dan nilainya lebih dari 0.



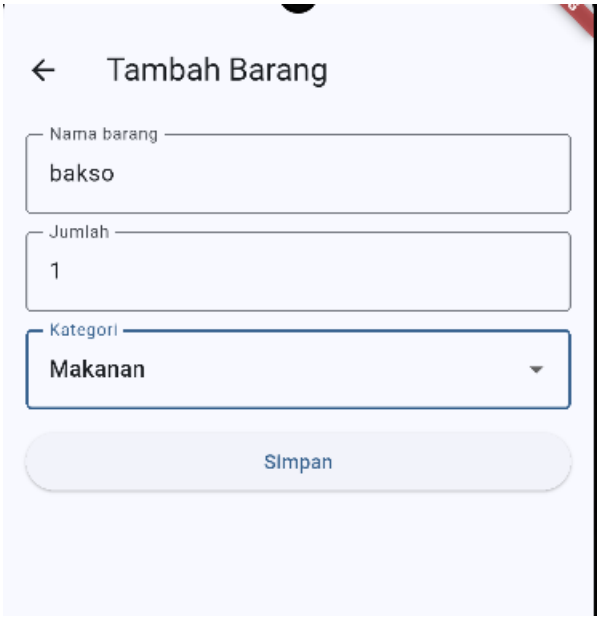
```
Widget build(BuildContext context) {
  final model = context.watch<BelanjaModel>();

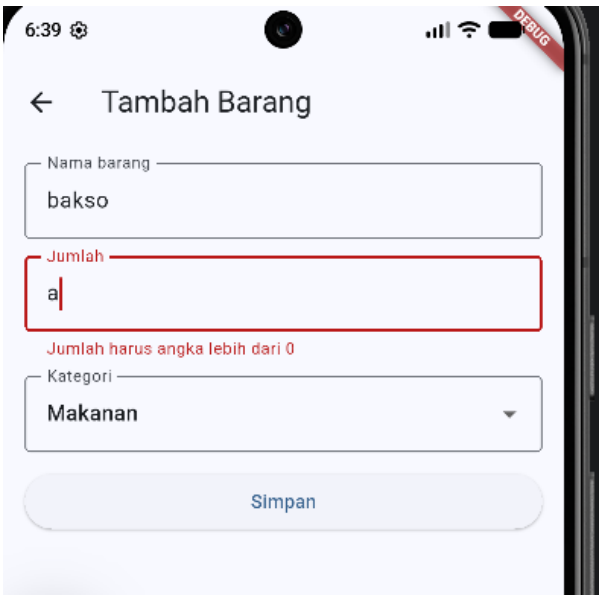
  return Scaffold(
    appBar: AppBar(
      title: Text(
        'Belum Dibeli (${model.jumlahBelumDibeli})',
      ), Text
    ), AppBar
    body: ListView.builder(
      itemCount: model.items.length,
      itemBuilder: (context, index) {
        final barang = model.items[index];

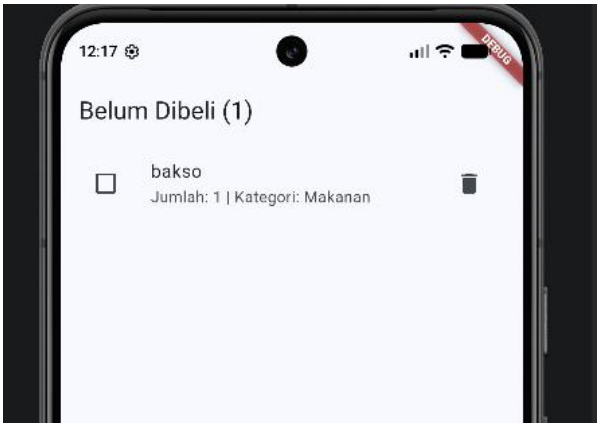
        return ListTile(
          leading: Checkbox(
            value: barang.sudahDibeli,
            onChanged: (_) {
              context.read<BelanjaModel>().toggle(index);
            },
          ), Checkbox
          title: Text(
            barang.nama,
            style: TextStyle(
              decoration: barang.sudahDibeli
                  ? TextDecoration.lineThrough
                  : null,
            ), TextStyle
```

Bagian ini digunakan untuk menampilkan seluruh barang yang tersimpan dalam BelanjaModel. context.watch() memungkinkan halaman memantau perubahan data sehingga tampilan diperbarui secara otomatis. Checkbox digunakan untuk menandai barang yang sudah dibeli, sedangkan context.read() digunakan untuk memanggil fungsi toggle() ketika checkbox diubah.

12. Hasil Tugas







13. Pertanyaan Refleksi

- Mengapa TextEditingController harus di-dispose?
  : Karena TextEditingController menyimpan resource dan mendaftarkan listener ke sistem (misalnya untuk memantau perubahan teks dan posisi kursor). Kalau tidak di-dispose, listener itu tetap hidup meskipun halaman sudah ditutup

- Kapan cukup menggunakan setState dan kapan menggunakan Provider?
  : setState cukup untuk state lokal yang hanya dipakai di satu widget/halaman dan tidak perlu dibagikan ke widget lain. Sementara provider digunakan untuk state global / dibagikan ke banyak widget atau halaman.

- Apa yang terjadi jika notifyListeners() tidak dipanggil?
  : UI tidak akan diperbarui. Data di dalam model mungkin sudah berubah (misalnya bertambah atau terhapus), tetapi karena notifyListeners() tidak dipanggil, Provider tidak memberi tahu widget yang mendengarkan, sehingga tampilan tetap menampilkan data lama.

- Mengapa onPressed menggunakan context.read() bukan context.watch()?
  : Karena onPressed adalah callback aksi, bukan bagian dari proses build. Disitu kita cuman ingin memanggil method pada model (seperti tambah, dan hapus), bukan memantau perubahan state. context.read() mengambil model sekali tanpa membuat widget ikut rebuild saat model berubah. Sedangkan context.watch() dipakai di dalam build ketika widget perlu mendengarkan perubahan dan rebuild otomatis.

14. Kesimpulan

Praktikum ini berhasil menerapkan input pengguna menggunakan TextField, pembuatan form dengan validasi, serta state management menggunakan ChangeNotifier dan Provider. Melalui aplikasi Daftar Belanja, data dapat digunakan pada beberapa halaman dengan state yang tersimpan pada satu model bersama.